LAB 7 : Graph

[CO5]

Instructions for students:

- Complete the following methods.
- You may use Java / Python to complete the tasks.
- DO NOT CREATE a separate folder for each task.

NOTE:

- **YOU CANNOT USE ANY OTHER DATA STRUCTURE OTHER THAN ARRAY UNLESS MENTIONED IN THE QUESTION.**
- **YOUR CODE SHOULD WORK FOR ANY VALID INPUTS.**

Python List, Negative indexing and append() is STRICTLY prohibited unless mentioned in the question

Lab Tasks: 4 tasks (0~1b)

Assignment Tasks: 6 tasks (2a~4b)

Total Assignment Mark: 6*5=30

Dear Students, you have been given instructions and driver code for the majority of the labs. For this lab no driver code will be given. You will develop everything (necessary functions, class, driver code, etc.) in your preferred language (Java or Python).

To solve the problems, draw a graph (**https://graphonline.top**) according to your preference. However, your graph must have a minimum of **7 vertices** and a minimum of **12 edges**. You can bring the graph you drew on the website to your code by going to Graph menu -> Adjacency Matrix menu and copying the matrix.

Note: Solve all the problems for both adjacency matrix representation and adjacency list representation. You can write different functions for each task to make your life easier.

Task 0: [Lab Task]

Represent the Graph you just drew in your code. Using both **Adjacency Matrix (Task0a)** and **Adjacency List (Task0b)**.

Task 1: [Lab Task]

Given an undirected, unweighted graph as input, write a function to find the vertex with maximum degree and return the degree of that vertex. Solve it for Adjacency Matrix **(Task1a)** & Adjacency List **(Task1b)**.

Task 2: [Assignment Task]

Given an undirected, edge-weighted graph as input, write a function to find the vertex whose sum of edge weights is maximum. Solve it for Adjacency Matrix **(Task2a)** & Adjacency List **(Task2b)**.

Task 3: [Assignment Task]

Solve Task 1 and Task 2 for directed, edge-weighted graphs considering only outgoing edges. Solve it for Adjacency Matrix **(Task3a)** & Adjacency List **(Task3b)**.

Task 4: [Assignment Task]

Write a function that will receive a directed weighted graph and convert it to an undirected weighted graph. Consider the weight of the edges will be unchanged. If there's a case where you have a bidirectional edge then you can sum up their weights. Solve it for Adjacency Matrix **(Task4a)** & Adjacency List **(Task4b)**.

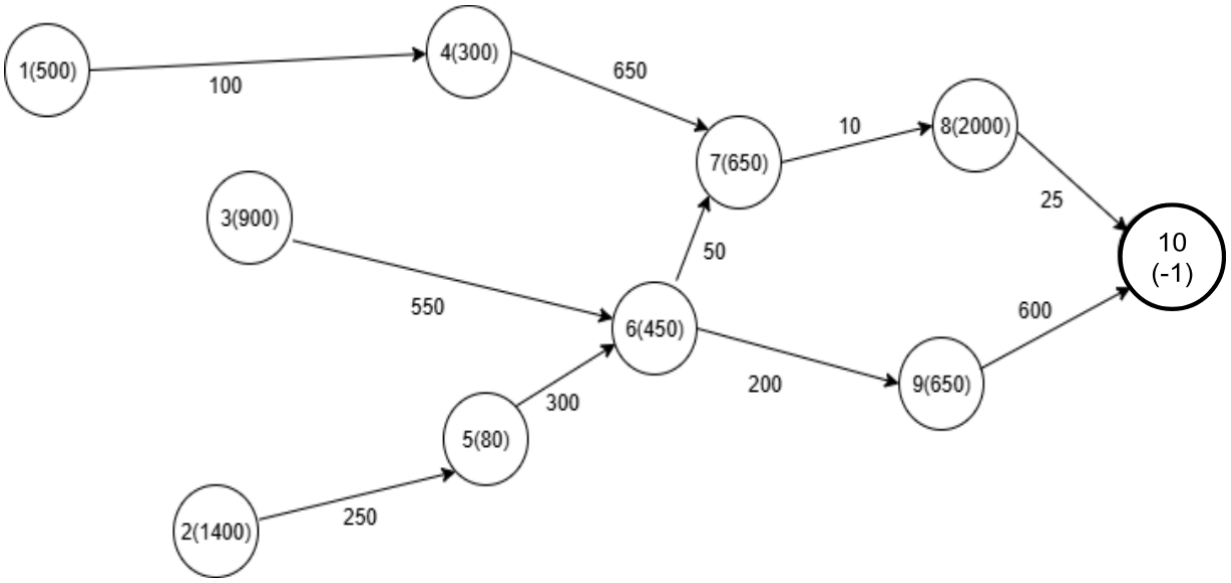
UNGRADED CODING TASK [Java Template,Python Template]

[Note: Try solving this coding question using Adjacency Matrix as well]

Luffy and Shanks start from given islands with initial power and aim to reach **Laugh Tale (vertex warLord power value is -1)**. Between islands, they fight a **Marine Officer (power given as edge weight)**. On each island, they fight a **Warlord (power given as vertex value)**. They always choose the next island with the **minimum (Marine Officer + Warlord) power**. Furthermore, stamina decreases after every single fight. If a pirate's stamina becomes less than the Warlord's power at any island, they cannot continue and fail.

Winner rules:

- If only one of them reaches Laugh Tale and the other can't → the one who reaches wins.
- If both reach Laugh Tale → the one with higher remaining stamina wins.
- If none reach Laugh Tale → Winner is no one.



[You can assume there's won't be any cyclic paths from the starting point to Laugh Tale]

[You can write helper functions if needed]

Python Method Signature
`def laughTale(wL, adjL, sSt, sP, lSt, lP):` `  #TODO`
Java Method Signature

`public static void laughTale(Integer[] warLords, Edge[] adjL, int sSt, int sP, int lSt, int lP){`
`  //TODO`
`}`

Warlord Powers, Sample Graph & Edge Class

warLords powers : [null, 500, 1400, 900, 300, 80, 450, 650, 2000, 650, -1]

Given Adjacency List :

0 : null
1 : <4,100> -> null
2 : <5,250> -> null
3 : <6,550> -> null
4 : <7,650> -> null
5 : <6,300> -> null
6 : <7,50> -> <9,200> -> null
7 : <8,10> -> null
8 : <10,25> -> null
9 : <10,600> -> null
10 : null

Given Java Edge Class

```
class Edge{
       int toV, weight;
       Edge next;
       public Edge(int t, int w){
              this.toV = t;
              this.weight = w;
       }
}
```

Given Python Edge Class

```
class Edge:
    def __init__(self, toV, weight):
        self.toV = toV
        self.weight = weight
        self.next = None
```

Sample Starting Vertex and Graph	Output
sSt = 2, sP = 7000, lSt = 1, lP = 3000	Winner is Shanks

Explanation

sSt: Shank's start from Island: 2
sP: Shanks's power: 7000
lSt: Luffy starts from Island: 1
lP: Luffy's power: 3000

Luffy's path: vertex 1 -> vertex 4 -> vertex 7 -> vertex 8
Luffy's power level: 3000-500-100-300-650-650-10=790 *[no more power left to reach vertex 10]*
Shanks's path: vertex 2 -> vertex 5 -> vertex 6 -> vertex 7 -> vertex 8 -> vertex 10
Shanks's power level: 7000-1400-250-80-300-450-50-650-10-2000-25= 1785.
So, Luffy cannot reach Laugh Tale and the **Winner is Shanks.**

Sample Starting Vertex and Graph	Output
sSt = 2, sP = 7000, lSt = 3, lP = 6500	Winner is Luffy

sSt: Shank's start from Island: 2
sP: Shanks's power: 7000
lSt: Luffy starts from Island: 3
lP: Luffy's power: 6500

Luffy's path: vertex 3 -> vertex 6 -> vertex 7 -> vertex 8 -> vertex 10
Luffy's power level: 6500-900-550-450-50-650-10-2000-25=1865

Shanks’s path: vertex 2 -> vertex 5 -> vertex 6 -> vertex 7 -> vertex 8 -> vertex 10
Shanks’s power level: 7000-1400-250-80-300-450-50-650-10-2000-25= 1785
Since 1865>1785. So, the **winner is Luffy**.